



Artificial Intelligence Practicum 2: Labelling

Authors:

Piotr Mateusz Bonar – NIU: 1759684

Tom Kurowski– NIU:1761154

Marek Tutka– NIU: 1733158

2025-05-19

Artificial Intelligence Practicum 2: Labelling	1
Introduction.....	3
Implemented Methods	3
KNN algorithm for Shape Classification	3
KNN functions	4
K-means algorithm for Color Classification	4
K-means functions.....	4
Analysis Functions	5
Qualitative Analysis	5
Color-based retrieval	5
Shape-based Retrieval	6
Combined Retrieval	7
Quantitative Analysis	8
K-Mean statistics	8
Accuracy assessment by shape	10
Accuracy assessment by color	11
Improvements	14
Improvements for KNN	14
Improvements for KMeans	16
Global conclusions	18

Introduction

The aim of this practicum is to build a system capable of labelling clothing items in images with two attributes: **color** and **shape**. The project is divided into three parts:

- **Part 1:** Implementation of the KMeans algorithm to detect dominant colors in images.
- **Part 2:** Implementation of the K-Nearest Neighbors (KNN) algorithm to classify the shape/type of clothing.
- **Part 3:** Evaluation and improvement of the combined system through qualitative and quantitative analyses.

By the end of the project, the system should be able to label an image like "Blue Jeans" or "Red Flip-Flops" with a reasonable degree of accuracy.

Implemented Methods

KNN algorithm for Shape Classification

We implemented the KMeans algorithm from scratch with the following capabilities:

- Different centroid initialization methods (first, random)
- Distance calculation using Euclidean norm
- Iterative update of centroids until convergence
- Measurement of Within-Class Distance (WCD) to evaluate clustering quality
- Automatic selection of best K using threshold-based improvement heuristic

Color tags were assigned using `get_colors()` based on centroid positions in RGB space.

Results:

- Dominant color clusters were accurately identified in most cases
- Visualizations showed clean segmentation for K=3 in the majority of images

KNN functions

`_init_train(train_data):`

Converts input RGB images to grayscale and flattens them into 1D vectors. The resulting training data matrix has shape $P \times D$, where P is the number of training images and D is the number of pixels.

`get_k_neighbours(test_data, k):`

For each test image, calculates its Euclidean distance to every training image, then retrieves the labels of the k nearest neighbors.

`get_class():`

Performs majority voting over the k nearest neighbors for each test image to determine the predicted class label.

`predict(test_data, k):`

Full inference pipeline combining neighbor retrieval and class prediction.

K-means algorithm for Color Classification

We implemented the KMeans algorithm from scratch for unsupervised color clustering. The system groups pixel colors into dominant clusters and assigns human-readable color labels based on centroid positions. This part was essential for recognizing and describing clothing colors without requiring manual annotation.

K-means functions

`_init_centroids():`

Initializes K centroids using either the 'first' K unique points or by selecting K random unique points.

`distance():`

Computes pairwise Euclidean distances between all data points and Centroids.

`get_labels():`

Assigns each point to the nearest centroid.

get_centroids():

Recomputes centroids as the mean of all assigned points.

converges():

Checks if centroids have stopped changing or maximum iterations is reached.

fit():

Runs the KMeans algorithm until convergence.

withinClassDistance():

Computes the average squared distance of all points to their assigned centroid.

find_bestK():

Uses percentage decrease in WCD to determine the optimal number of clusters.

get_colors():

Maps RGB centroids to predefined color labels using a probability-based function.

Analysis Functions

Qualitative Analysis

For the Qualitative functions, we have implemented:

- Color-based retrieval (retrieve_by_color)
- Shape-based Retrieval (retrieve_by_shape)
- Combined Retrieval (retrieve_combined)

Color-based retrieval

Requires:

- List of images – list of images
- Color tags – list of lists of [color,percentage]
- Query colors – list of colors that we want to search for
- Ground truth (optional) - list of correct color tags
- Color Percentages (optional) - list of percentages for color

Returns:

- List of images
- Additional statistics [precision, recall, number of found images]

Example search for colors: ['Red', 'Blue'] with Kmeans(K=6) gave us the following results:

Found 28 images containing ['Red', 'Blue'] (of 17 in ground truth)
 Precision: 21.4%, Recall: 35.3%
 F1-Score: 26.7%

Example search for colors: ['Red', 'Blue'] with Kmeans(K=3) gave us the following results:

Found 4 images containing ['Red', 'Blue'] (of 17 in ground truth)
 Precision: 50.0%, Recall: 11.8%
 F1-Score: 19.0%

We can see that the model struggles with finding red and blue combinations, increasing the K value gave a slightly improvement of the F1-score, it finds more images, however the accuracy still remains low

Shape-based Retrieval

Requires:

- List of images – list of images
- Predicted shape – list of labels assigned for each image by our classifier
- Query shape – name of the shape that we want to look for
- True_tags – list of the ground truth, for statistics comparison

Returns:

- Matched images – list of images that matched the query shape
- Additional statistics [precision, recall, number of found images]

We have run the function for “Jeans” as example and the results are following:

Found 97 images labeled as 'Jeans' (of 93 in ground truth)
 Precision: 92.8%,
 Recall: 96.8%

Which shows that the shape retrieval works good, it finds most of the images with low false positive and negative rate.

It is also confirmed by the quantitative analysis of the shape accuracy showed in the “Accuracy assessment by color” section.

Combined Retrieval

Requires:

- List of images
- Predicted Shape tags – List of tags
- Predicted color tags – List of tags
- Query shape – String of the shape to look for
- Query colors – List of colors to look for
- List of shapes confidence (Optional)
- List of colors percentage (Optional)

Returns:

- List of found images

Shows:

- Canva showing matched images and it's names

The function has optional arguments for shapes confidence and colors percentage in the image. When they are not provided, the function performs binary matching, meaning that they are treated equally and results are returned without any sorting.

When those arguments are provided, then the results are sorted based on their score. The total score of the match is calculated as $(\text{shape_score} * \text{color_percentage})$. Thanks to that, we can show the best matching images (with color and shape) in the top of the Canva.

The example search for “Blue Jeans” (without scores, therefore the order of the image is original as in the provided list) is shown in the below picture.

We can see that the results are good, most of the images are in fact “Blue jeans”,



Image showing matches for “Blue Jeans”

Quantitative Analysis

For the Quantitative functions we have implemented:

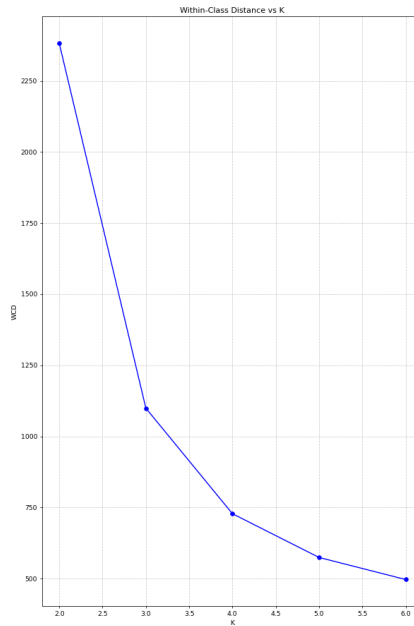
- K-Means Statistics (kmean_statistics)
- Accuracy assessment by shape (get_shape_accuracy)
- Accuracy assessment by color (get_color_accuracy)

K-Mean statistics

Currently the K-Mean statistics give us 3 main metrics:

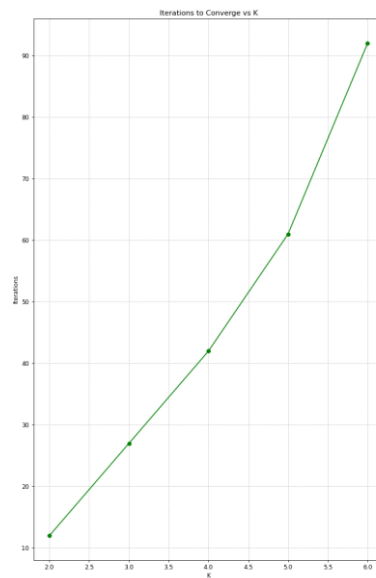
- Within-class distance
- Time to converge
- Number of iterations to converge

Below image show the WCD for K, we see that the WCD indeed is decreasing when we increase number of centroids.



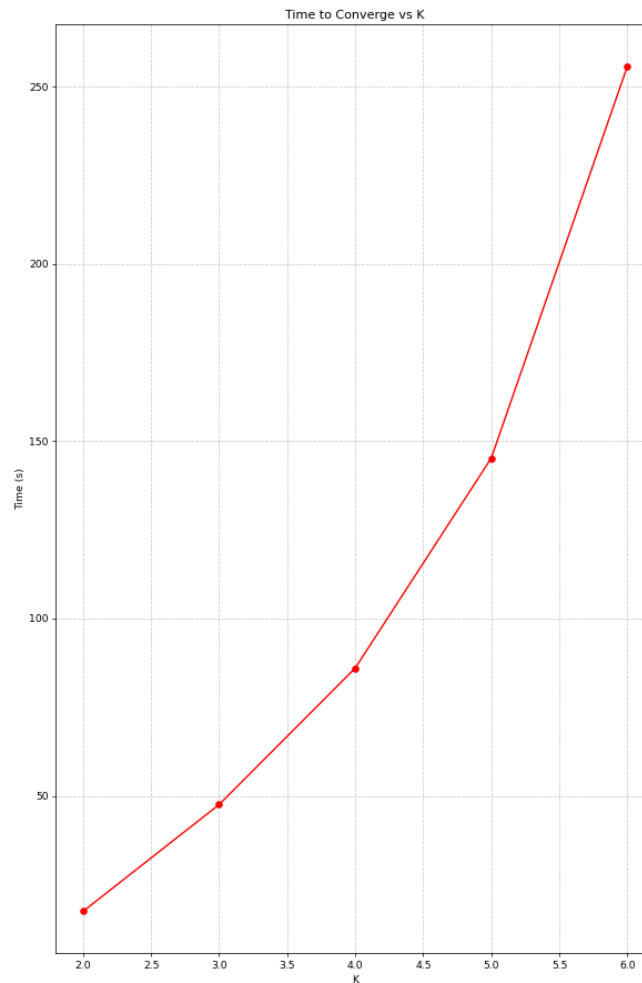
Plot of the Within-Class distance with K number of centroids

The Below image show is the number of iterations to converge, we see that with increasing number of centroids (K) it needs more iterations to find the optimal centroids, which is expected.



Plot of the iterations to converge with K number of centroids

The below image shows us the time to converge based on K, similar to the previous plot, we see that with increasing number of centroids, the time is increasing, which is expected.



Plot of time to converge for K number of centroids

Accuracy assessment by shape

Requires:

- List of predictions tags
- List of ground truth tags

Returns:

- Accuracy score – Percentage of correctly predicted tags

We are computing percentage of the exact shape matches of the predictions and ground truth (correct / total)

After running the shape test with Knn(K=3) for all images, we have received 91.5% accuracy,

Test-set shape accuracy: 91.5%

	Precision	Recall	F1-score
Dresses	90.7%	94.6%	92.6%
Flip Flops	75.7%	89.7%	82.1%
Handbags	97.8%	96.8%	97.3%
Heels	95.3%	82.8%	88.6%
Jeans	92.8%	96.8%	94.7%
Sandals	87.8%	84.0%	85.9%
Shirts	97.9%	96.9%	97.4%
Shorts	91.0%	90.0%	90.5%
Socks	98.9%	92.6%	95.6%

After running the shape test with Knn(K=6) for all images, we have received:

Test-set shape accuracy: 89.8%

	Precision	Recall	F1-score
Dresses	90.8%	95.7%	93.2%
Flip Flops	73.2%	84.5%	78.5%
Handbags	94.7%	95.7%	95.2%
Heels	84.6%	77.8%	81.1%
Jeans	95.7%	94.6%	95.1%
Sandals	89.5%	81.9%	85.6%
Shirts	98.9%	95.9%	97.4%
Shorts	90.0%	90.0%	90.0%
Socks	93.5%	92.6%	93.0%

Accuracy assessment by color

Requires:

- List of color prediction tags

- List of color ground truth tags

Returns:

- Average Jaccard score
- Per color statistics - List of lists [precision, recall, f1-score]

Shows:

- Confusion Matrix

The function compares predictions with the ground truth producing the per-color statistics as: precision, recall, f1-score. The overall accuracy is calculated as the average Jaccard score.

The formula to calculate the Jaccard score is the following:

$$\text{Jaccard} = (\text{Number of elements in both sets}) / (\text{Number of elements in either set})$$

$$= |A \cap B| / |A \cup B|$$

Example of Jaccard score:

If true colors are {"Red", "Blue", "Black"}

And predicted colors are {"Red", "Blue", "White"}

Intersection: {"Red", "Blue"} (2 elements)

Union: {"Red", "Blue", "Black", "White"} (4 elements)

Jaccard similarity = $2/4 = 0.5$ (50%)

In the confusion matrix, when the color was predicted correctly, then 1 point is added to the in the diagonal position (for example matrix[true_black,predicted_black]). When the true color tag is not in the predicted tags, the point is divided by total number of predictions and assigned to the matrix[true_missing,predicted] for every predicted color.

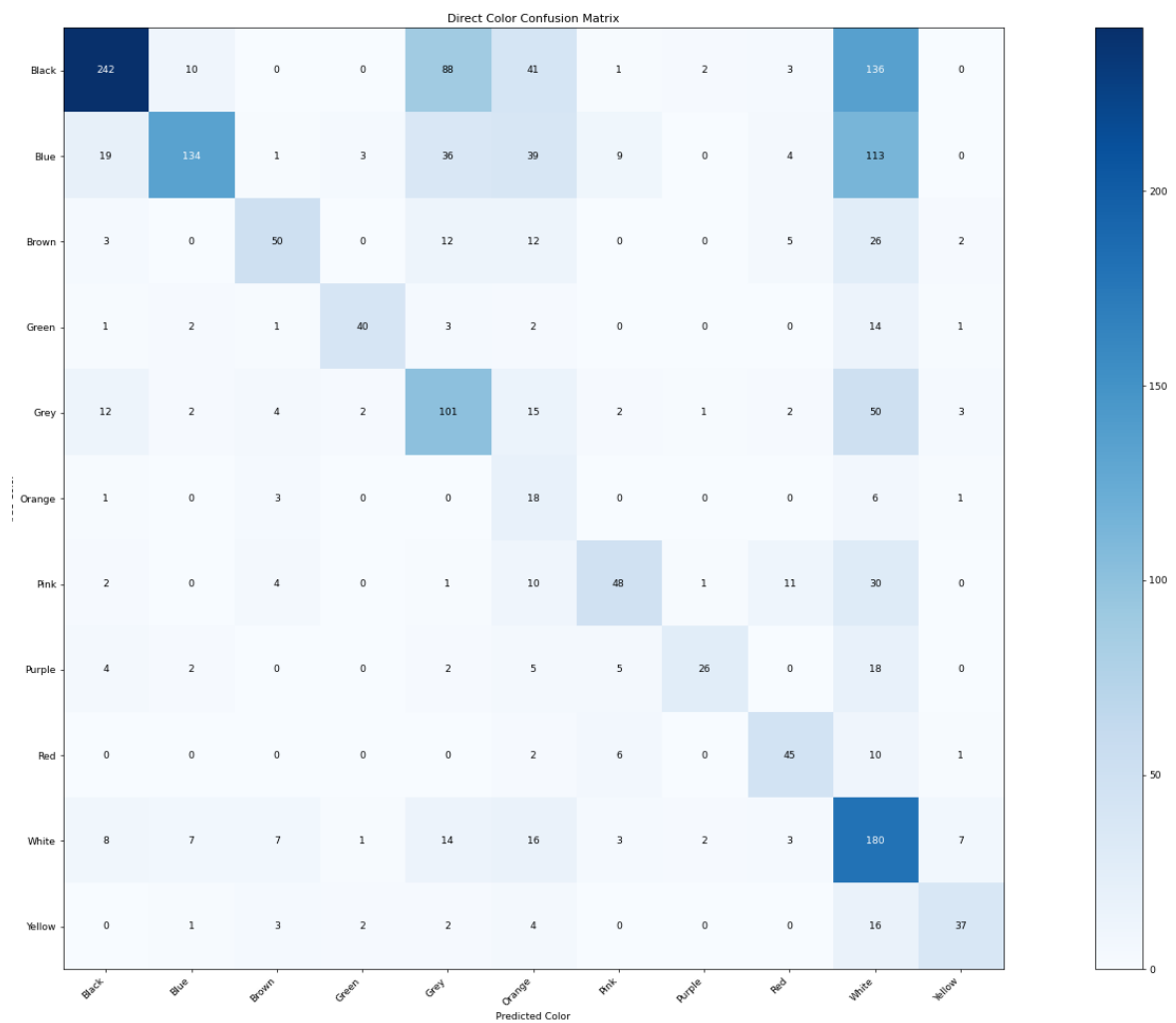
We have tested Kmeans(K=6,threshold=0.2) and Kmeans(K=6,threshold=0.1). The overall accuracy improved from 37.8% (using threshold=0.2) to 48.4% (using threshold=0.1), which still is not really satisfying. From the statistics, we can see that the Black and Blue are the easiest for our model. The hardest is the orange, we see that we predict small amount and even if we predict, the accuracy is almost 0.

Results with the Kmeans(K=6, threshold=0.10)

--- Color Accuracy Statistics ---

Overall accuracy: 48.4%

	Precision	Recall	F1-score
Black	0.81	0.64	0.71
Blue	0.88	0.55	0.67
Brown	0.56	0.31	0.40
Green	0.80	0.31	0.44
Grey	0.58	0.52	0.55
Orange	0.08	0.12	0.10
Pink	0.83	0.38	0.53
Purple	0.40	0.25	0.31
Red	0.40	0.40	0.40
White	0.24	0.74	0.37
Yellow	0.50	0.20	0.29



Results with Kmeans(K=6, threshold=0.2)

--- Color Accuracy Statistics ---

Overall accuracy: 37.8%

	Precision	Recall	F1-score
Black	0.69	0.90	0.78
Blue	0.76	0.85	0.80
Brown	0.46	0.61	0.52
Green	0.74	0.64	0.69
Grey	0.26	0.92	0.41
Orange	0.07	0.93	0.14
Pink	0.48	0.79	0.60
Purple	0.57	0.64	0.60
Red	0.37	0.81	0.51
White	0.21	1.00	0.35
Yellow	0.65	0.75	0.70

Improvements

The goal of this part of our practicum was to implement some improvements/changes to our implemented KNN and KMeans algorithm.

Improvements for KNN

As an improvement to the standard K-Nearest Neighbors (KNN) implementation, we experimented with using Manhattan distance (also known as L1 distance) instead of the

commonly used Euclidean distance (L2). The goal of this modification was to assess whether a different distance metric could better capture the similarity between grayscale image representations of clothing items.

In the original version, the similarity between test and training images was calculated using the Euclidean distance in a high-dimensional pixel space. We modified the KNN class to accept a `distance_metric` parameter, allowing a dynamic switch between Euclidean and Manhattan distances. The Manhattan distance computes the absolute difference across all dimensions and is known to be more robust to outliers in certain contexts.

We evaluated both metrics across different values of k and measured the shape classification accuracy on the test set. This can be seen in the following table.

k	Accuracy (Euclidean)	Accuracy (Manhattan)
1	92.7 %	91.4 %
2	92.7 %	91.4 %
3	91.5 %	90.5 %
4	91.9 %	90.7 %
5	90.1 %	90.2 %
6	89.8 %	91.1 %
7	90.0 %	90.4 %

Although Manhattan distance is known to be more robust to noise and outliers, our experiments showed that it consistently performed slightly worse than Euclidean distance across the lower values of k (especially 1-4).

One possible reason is the high dimensionality of the input data: each image was flattened into a 4800-dimensional vector, where Euclidean distance tends to preserve better discriminative power than Manhattan distance. In high-dimensional spaces, Manhattan distances can become less informative because the differences across many dimensions accumulate evenly, making it harder to distinguish between similar and dissimilar samples. Additionally, Euclidean distance penalizes larger differences more strongly due to squaring, which may help emphasize meaningful pixel-level variations in grayscale images of clothing. Since our dataset was relatively clean and continuous, the robustness of Manhattan distance likely offered no advantage, leading to overall slightly lower accuracy.

Improvements for KMeans

To improve the performance of the KMeans-based color classification, we implemented two additional heuristics to determine the optimal number of clusters (K), beyond the default Within-Class Distance (WCD) method used in Part 1 of the project.

The first heuristic is the Inter-Class Distance (ICD), which measures the average pairwise distance between all centroids. The idea behind this metric is that well-separated clusters (with higher inter-class distances) tend to correspond to better-defined color groupings. By maximizing the ICD, we aim to find a value of K that results in clearly separated color regions.

The second heuristic we implemented is the Fisher Criterion, which balances intra-cluster compactness with inter-cluster separation. It is defined as the ratio between the Within-Class Distance and the Inter-Class Distance. A lower Fisher score indicates that the clusters are compact and well separated, which is desirable in unsupervised classification tasks.

We integrated both heuristics into the `find_bestK_by_heuristic` method of our KMeans class and systematically compared the results to the WCD-based method. This allowed us to observe how different heuristics influence the choice of K, and how this, in turn, affects the overall color labeling accuracy on the test set.

To compare the different heuristics for selecting the optimal number of clusters in KMeans color labeling, we applied each heuristic individually to every image in the test set. For each image, we determined the best number of clusters (K) using one of the heuristics either within-class distance (WCD), inter-class distance, or the Fisher criterion. After selecting the optimal K, we performed KMeans clustering on the image's pixel data and extracted the resulting centroids. These centroids were then mapped to color labels, which were compared against the ground truth to evaluate classification accuracy. This process allowed us to assess how effectively each heuristic guided the selection of K in terms of overall color prediction performance. The obtained accuracies for the different methods can be seen in the following tables (first table with `km_init = "random"`, second table with `km_init = "first"`).

Heuristic	Color Accuracy
WCD	35.7 %
Inter-Class	33.8 %
Fisher	35.7 %

Heuristic	Color Accuracy
WCD	37.8 %

Inter-Class	35.0 %
Fisher	37.7 %

The comparison shows that the choice of `km_init` has a notable impact on color classification accuracy. When using `km_init = first`, all heuristics performed better than with random initialization, likely due to more consistent and deterministic centroid starting positions.

Among the heuristics, both WCD and Fisher achieved the highest accuracy with `km_init = first`, reaching 37.8 % and 37.7 % respectively. This suggests that both compactness (WCD) and the balance between compactness and separation (Fisher) are effective criteria for selecting the number of clusters in this setting.

In contrast, the Inter-Class Distance heuristic performed the worst across both initializations, indicating that maximizing centroid separation alone is not sufficient for good clustering performance in this task. Overall, WCD and Fisher are more reliable heuristics for guiding the choice of `K` in color-based image clustering.

As a second improvement, we focused on refining the selection of the optimal number of clusters `K`. In Part 1 of the project, we used a fixed threshold of 20% improvement in within-class distance (WCD) to decide when to stop increasing `K`. In this part, we investigated whether varying this threshold could lead to better results. We constructed a table that evaluated different values of `K` (ranging from 2 to 8) across multiple threshold settings. By analyzing the resulting WCD values and corresponding classification accuracies, we were able to identify more effective threshold configurations, leading to improved control over the clustering process and more accurate color classification. In the following a table of the different settings can be seen.

K\th r	0%	5%	10%	15%	20%	25%	30%	35%	40%	45%	50%
K=2	39.0	39.3	39.8	39.2	39.0	40.8	41.1	40.8	37.0	36.0	36.3
K=3	28.9	31.7	37.4	36.9	36.7	36.1	34.4	31.9	30.6	30.5	30.6
K=4	26.9	34.7	40.0	39.4	36.0	34.9	33.5	31.2	30.5	30.3	30.3
K=5	27.0	40.7	41.4	39.6	36.6	33.9	33.9	32.9	33.0	33.0	33.0
K=6	23.9	43.1	48.4	43.4	41.0	39.3	38.8	38.0	38.0	38.0	38.0
K=7	24.0	43.2	43.2	41.3	39.3	38.8	38.3	38.3	38.3	38.3	38.3
K=8	23.9	44.7	43.5	41.2	40.6	40.0	39.7	39.7	39.7	39.7	39.7

Global conclusions

This practicum successfully demonstrated how classic machine learning techniques can be applied to the task of image labelling. By integrating **KMeans** for color detection and **KNN** for shape classification, we built a two-stage labelling pipeline that performs reliably on a variety of clothing images.

Achievements:

- Accurate detection of dominant colors using unsupervised clustering.
- Effective shape classification using a lightweight and interpretable classifier.
- A modular design that allows qualitative retrieval (color, shape, and combined) and quantitative evaluation.
- Visualization and testing infrastructure enabling easy interpretability of results.

Key Insights:

- KMeans is sensitive to centroid initialization and the choice of K; our experiments showed that using randomized initialization and adaptive thresholding for BestK provided more stable results.
- KNN, while simple, performs robustly when combined with basic feature extraction; however, further performance could be achieved using more advanced distance metrics or feature engineering.

Challenges and Limitations:

- KMeans may group colors inconsistently when background interference is strong.
- Shape classification relies heavily on image resolution and grayscale uniformity.
- No segmentation was performed, so unrelated background content may affect predictions.

Applications:

- Visual clothing search engines.
- E-commerce product tagging and filtering.
- Organizing and indexing fashion datasets.

In conclusion, the labelling system developed is functional, extensible, and serves as a strong baseline for further exploration into more advanced computer vision methods.

